

东北石油大学

本科生课程设计

基于 Spring Boot 的 服装私人定制网站设计与实现

学 生 姓 名：朱浩云

学 号：250701240212

专 业：网络空间安全

指 导 教 师：赵娅 副教授

完 成 日 期：2025 年 12 月

课程设计任务书

课程名称：Java Web 应用开发

题 目：基于 Spring Boot 的服装私人定制网站设计与实现

学 生：朱浩云（250701240212）

专 业：网络空间安全

指导教师：赵娅 副教授

一、课程设计目的

本课程设计旨在综合运用 Java Web 开发技术，完成一个具有实际应用价值的服装私人定制网站系统。通过本项目的开发，学生需要掌握 Spring Boot 框架的核心原理与使用方法，熟练运用 JPA/Hibernate 进行对象关系映射与数据库操作，理解 B/S 架构的设计思想，掌握前端 HTML/CSS/JavaScript 基础技术，并能够运用 Maven 进行项目构建与依赖管理。同时，通过完整的项目实践，培养学生的系统分析与设计能力、代码编写与调试能力、以及软件工程文档撰写能力。

二、课程设计内容与要求

1. 需求分析：分析服装私人定制业务流程，明确系统功能性需求与非功能性需求，包括用户管理、面料管理、款式管理、量体数据管理、订单管理和后台管理等核心功能模块。

2. 系统设计：完成系统架构设计、功能模块设计、数据库设计（E-R 图设计与数据表结构设计），确定技术选型方案，遵循分层解耦、单一职责、安全优先等设计原则。

3. 功能实现：基于 Spring Boot 4.1.0 框架与 Java 25 语言，使用 JPA/Hibernate ORM 框架与 H2 文件数据库，实现用户注册登录、面料浏览筛选、款式浏览搜索、量体数据录入、用户下单、订单状态推进等全部功能。前端采用 HTML/CSS/JavaScript 技术，通过 RESTful API 与后端通信。

4. 安全实现：采用 BCrypt 密码加密算法保障用户密码安全，使用 Session 机制进行用户认证，通过拦截器实现 URL 级别的权限控制，防止未授权访问与越权操作。

5. 系统测试：编写测试用例，对系统的功能正确性、安全性、兼容性进行全面测试，确保系统稳定可靠运行。

6. 文档撰写：按照学校规定的格式要求，撰写完整的课程设计论文，包括摘要、目录、正文各章节、结论与参考文献等。

三、时间安排

第1周：需求分析与系统设计，完成需求分析文档与系统设计文档。

第2周：数据库设计与后端核心功能开发，完成实体类、Repository 接口、Service 层与 Controller 层代码编写。

第3周：前端页面开发与系统集成，完成 HTML/CSS/JavaScript 前端页面，实现前后端联调。

第4周：系统测试、优化与论文撰写，完成测试报告与课程设计论文。

四、预期成果

1. 可运行的服装私人定制网站系统源代码（含 Maven 构建脚本）。
2. 课程设计论文一份（不少于 45 页），包含完整的系统分析与设计文档。
3. 系统测试报告，覆盖功能测试、安全测试与兼容性测试。

摘 要

随着服装行业数字化转型的加速推进和消费者个性化需求的日益增长，传统的线下服装定制模式已难以满足现代消费者对便捷性、透明度和个性化体验的追求。本文设计并实现了一个基于 Spring Boot 框架的服装私人定制网站系统，旨在为用户提供便捷的线上服装定制服务。

系统采用 B/S 架构，后端基于 Spring Boot 4.1.0 框架与 Java 25 语言开发，使用 JPA/Hibernate 进行对象关系映射，采用 H2 文件数据库实现数据持久化存储，通过 Maven 进行项目构建与依赖管理。前端使用 HTML/CSS/JavaScript 技术栈，通过 RESTful API 与后端进行数据交互。系统实现了用户注册登录、面料浏览与筛选、款式浏览与搜索、量体数据管理、订单创建与查询、后台管理等核心功能模块。在安全方面，系统采用 BCrypt 密码加密算法保障用户密码安全，使用 Session 机制进行用户认证，并通过拦截器实现 URL 级别的权限控制，有效防止未授权访问与越权操作。

系统采用分层架构设计，将表现层、控制层、业务层和数据访问层进行解耦，提高了代码的可维护性和可扩展性。订单管理模块实现了从待确认到已完成的全流程状态推进机制，保障了定制生产流程的规范性。经过功能测试、安全测试和兼容性测试，系统各功能模块运行稳定，达到了预期设计目标，具有一定的实用价值和推广意义。

关键词：Spring Boot；服装定制；JPA；H2 数据库；B/S 架构

Abstract

With the acceleration of digital transformation in the apparel industry and the growing demand for personalization among consumers, the traditional offline clothing customization model can no longer meet modern consumers' pursuit of convenience, transparency, and personalized experience. This paper designs and implements a clothing customization website system based on the Spring Boot framework, aiming to provide users with convenient online clothing customization services.

The system adopts a B/S architecture, with the backend developed based on the Spring Boot 4.1.0 framework and Java 25, using JPA/Hibernate for object-relational mapping, H2 file database for data persistence, and Maven for project building and dependency management. The frontend utilizes HTML/CSS/JavaScript technology stack and communicates with the backend through RESTful APIs. The system implements core functional modules including user registration and login, fabric browsing and filtering, style browsing and searching, measurement data management, order creation and querying, and backend management. In terms of security, the system employs the BCrypt password encryption algorithm to ensure password security, uses Session mechanism for user authentication, and implements URL-level access control through interceptors to effectively prevent unauthorized access and privilege escalation.

The system adopts a layered architecture design, decoupling the presentation layer, control layer, business layer, and data access layer to improve code maintainability and scalability. The order management module implements a full-process status advancement mechanism from pending to completed, ensuring the standardization of the customization production process. Through functional testing, security testing, and compatibility testing, all functional modules of the system operate stably and achieve the expected design objectives, demonstrating practical value and promotional significance.

Key words: Spring Boot; Clothing Customization; JPA; H2 Database; B/S Architecture

目 录

摘 要.....	I
Abstract.....	II
第 1 章 概述.....	1
1.1 系统开发背景及意义.....	1
1.2 国内外研究现状.....	2
1.3 主要研究内容.....	3
第 2 章 相关技术及工具介绍.....	5
2.1 Spring Boot 框架.....	5
2.2 JPA 与 Hibernate.....	6
2.3 H2 数据库.....	7
2.4 HTML/CSS/JavaScript.....	8
2.5 Maven 构建工具.....	9
第 3 章 需求分析.....	11
3.1 功能性需求分析.....	11
3.2 非功能性需求分析.....	13
第 4 章 系统设计.....	15
4.1 设计原则.....	15
4.2 系统架构设计.....	15
4.3 功能模块设计.....	17
4.4 数据库设计.....	19
第 5 章 功能实现.....	24
5.1 登录系统功能.....	24
5.2 信息展示功能.....	26
5.3 信息查询功能.....	28
5.4 信息改动功能.....	30
第 6 章 系统测试.....	33
6.1 测试方案.....	33

6.2 测试结果.....	35
结论.....	38
参考文献.....	39

第1章 概述

1.1 系统开发背景及意义

服装是人类生活的基本需求之一，随着社会经济的发展和人民生活水平的提高，消费者对服装的需求已从基本的遮体保暖功能转向追求个性化、品质化和品牌化。在这一趋势下，服装定制作为一种能够充分满足消费者个性化需求的服务模式，正受到越来越多的关注。传统的服装定制主要依赖线下实体店面，消费者需要亲自到店量体、选款、选面料，整个过程耗时长、信息不对称、价格不透明，且受限于地理位置，难以实现规模化发展[1]。

近年来，随着互联网技术的飞速发展和电子商务的普及，服装行业正经历着深刻的数字化转型。C2M（Customer to Manufacturer，消费者直达工厂）模式的兴起，为服装定制行业的线上化发展提供了新的契机。C2M模式通过互联网平台直接连接消费者与生产企业，省去了中间分销环节，不仅降低了定制成本，还缩短了交付周期。据行业研究报告显示，中国服装定制市场规模已突破千亿元，并保持着年均两位数的增长率，市场前景广阔[2]。

然而，当前市场上的服装定制平台仍存在诸多不足。一方面，部分平台仅提供简单的在线选款功能，缺乏量体数据管理和面料选择等深度定制能力，无法真正满足消费者的个性化需求。另一方面，许多平台的订单流程不透明，消费者难以实时了解定制进度，用户体验有待提升。此外，中小型服装定制企业缺乏专业的信息化管理系统，订单管理、面料管理、量体数据管理等环节仍依赖人工操作，效率低下且容易出错[3]。

基于上述背景，开发一个功能完善、操作便捷的服装私人定制网站系统具有重要的现实意义。首先，线上平台能够打破时间和空间的限制，消费者可以随时随地浏览面料、选择款式、录入量体数据并下单，极大提升了定制服务的可及性。其次，系统化的订单管理流程能够实现从下单到完成的全流程跟踪，提高流程透明度，增强消费者信任。再次，通过数字化管理面料库存、款式信息和客户量体数据，可以有效提升企业的运营效率和管理水平。最后，本系统的开发也是对 Spring Boot 框架、JPA 持久化技术、B/S 架构设计等 Java Web 开发技术的综合实践，具有较强的教学价值和技术参考意义。

从技术发展趋势来看，近年来 Java Web 开发技术体系日趋成熟，Spring Boot 框架的推出极大降低了企业级应用的开发门槛，JPA 规范的普及使得数据持久层的开发更加规范化，前后端分离架构的推广提升了系统的可维护性和可扩展性。这些技

术进步为开发轻量级、高效率的服装定制系统提供了坚实的技术基础。与此同时，H2 等嵌入式数据库的成熟使得小型项目无需部署独立的数据库服务即可实现数据持久化，进一步降低了系统的部署和运维成本。本系统正是基于上述技术体系构建，旨在探索一种适合中小型服装定制企业的轻量级信息化解决方案。

1.2 国内外研究现状

1.2.1 国内研究现状

国内服装定制行业的数字化发展正处于从传统模式向 C2M 模式转型的关键时期。在技术层面，三维人体重建技术、AI 辅助设计技术和虚拟试衣技术等前沿技术正逐步应用于服装定制领域。刘艺舒等人研究了面向服装个性化定制的多视角轮廓三维人体快速重建方法，通过多视角图像采集与轮廓提取实现了人体三维模型的快速构建，为线上量体提供了技术支撑[1]。王建萍等人基于 RFID 技术研究了大制模服装定制生产信息系统，通过 RFID 标签实现生产流程的实时追踪与信息管理，提升了定制生产效率[2]。李敏等人设计了基于远程服装定制的人体数据测量系统，实现了用户远程自助量体与数据上传功能[3]。

在应用层面，国内涌现了一批服装定制平台，如量品、衣邦人、红领等，这些平台在 C2M 模式探索方面取得了显著进展。然而，这些平台多聚焦于特定品类（如衬衫、西装）的定制，且系统架构复杂、开发成本高，不太适合中小型服装定制企业使用。此外，近年来新中式服装风潮的兴起，进一步推动了服装定制市场的多元化发展，对定制平台的面料管理、款式管理和量体数据管理能力提出了更高要求[4]。

从市场规模来看，国内服装定制市场已突破千亿元规模，年增长率保持在 15% 以上。C2M（Customer to Manufacturer）模式的兴起有效降低了定制服装的价格门槛，使得定制服务从高端小众市场逐步走向大众化。然而，当前国内服装定制行业仍面临诸多挑战：一是专业人才断层，裁缝师傅年龄结构老化，年轻从业者数量不足；二是消费者认知偏差，部分消费者对线上定制的量体准确性和成品质量存在顾虑；三是信息化水平参差不齐，大量中小型定制门店仍采用纸质记录和口头沟通的方式管理订单，效率低下且容易出错。这些痛点为本系统的开发提供了明确的需求导向。

在技术研究方向上，国内学者还积极探索了智能化定制方案。基于 BP 神经网络的服装定制尺寸设计方法能够根据用户的身高、体重等基本参数智能推荐各部位尺寸，减少人工量体的误差。三维交互式服装设计系统则允许用户在虚拟环境中实时预览定制效果，调整面料纹理和款式细节。这些技术探索为服装定制系统的功能

创新提供了丰富的参考，但由于技术复杂度高、硬件要求苛刻，短期内难以在中小型平台中大规模推广。本系统选择以基础功能完善为目标，采用成熟稳定的 Web 技术栈，优先保障系统的可用性和可靠性。

1.2.2 国外研究现状

国外服装定制行业的数字化发展起步较早，数字化普及程度较高。在北美市场，定制服装市场份额已达到约 34%，消费者对线上定制的接受度和满意度较高。虚拟试衣技术在国外已趋于常态化应用，消费者可以通过 AR/VR 技术在虚拟环境中预览定制效果，大幅降低了定制的不确定性。此外，可持续时尚理念在欧美市场深入人心，越来越多的定制平台开始关注环保面料的选择和生产过程的碳足迹管理，推动服装定制行业向绿色可持续方向发展[5]。

在技术架构方面，国外主流服装定制平台普遍采用微服务架构和云原生技术，具备高可用性和弹性伸缩能力。然而，这些技术方案对于中小型企业 and 教学实践而言过于复杂，需要投入大量的开发和运维资源。因此，研究基于轻量级框架（如 Spring Boot）的服装定制系统，对于降低开发门槛、促进技术普及具有重要价值。

从区域市场格局来看，全球服装定制市场呈现出明显的区域分化特征。北美市场以 34% 的份额占据全球最大市场份额，消费者对个性化定制的需求旺盛，支付意愿强。欧洲市场注重工艺传承和面料品质，定制服装在意大利、英国和法国等传统时尚强国具有深厚的文化根基。亚洲市场虽然起步较晚，但增长速度最快，特别是中国和印度等人口大国的定制需求正在快速释放。这种区域分化意味着服装定制系统需要具备良好的本地化适配能力，支持多语言、多币种和差异化业务规则。本系统虽然聚焦于中国市场，但其模块化设计为未来扩展国际化功能预留了空间。

此外，国外在服装定制标准化方面也取得了显著进展。ISO 等国际组织制定了多项服装尺寸标准和量体数据交换标准，为不同平台之间的数据互通奠定了基础。日本在跨境电商和全球化定制方面走在前列，多家平台已实现跨国量体数据传输和远程定制生产。这些标准化实践对于本系统的量体数据字段设计具有参考价值，本系统的 Measurement 实体涵盖了 14 项标准化身体尺寸，基本覆盖了上衣和裤装定制的主要测量点。

1.3 主要研究内容

本课题以服装私人定制业务为背景，设计并实现一个基于 Spring Boot 框架的服装私人定制网站系统。主要研究内容包括以下六个方面：

（1）系统需求分析：深入分析服装定制业务流程，明确系统的功能性需求与非功能性需求，包括前台用户功能和后台管理功能，为系统设计与实现奠定基础。

（2）系统架构设计：采用 B/S 架构与分层设计思想，设计表现层、控制层、业务层、数据访问层和数据层五层架构体系，确定技术选型方案，保障系统的可维护性与可扩展性。

（3）数据库设计：根据业务需求设计 E-R 模型，建立管理员表、用户表、面料表、款式表、量体数据表和订单表共 6 张数据表，定义实体间的关联关系，设计订单状态流转机制。

（4）核心功能实现：基于 Spring Boot 4.1.0 框架与 Java 25 语言，实现用户注册登录、面料管理、款式管理、量体数据管理、订单管理和后台管理等核心功能模块，前端采用 HTML/CSS/JavaScript 技术并通过 RESTful API 与后端通信。

（5）安全机制实现：采用 BCrypt 密码加密算法保障密码安全，使用 Session 机制进行用户认证，通过拦截器实现 URL 级别的权限控制，在量体数据管理中实现数据归属校验以防止越权操作。

（6）系统测试与优化：设计测试用例，对系统的功能正确性、安全性和兼容性进行全面测试，根据测试结果进行问题修复与系统优化，确保系统稳定可靠运行。

第2章 相关技术及工具介绍

本章对系统开发过程中所采用的关键技术与工具进行介绍，包括 Spring Boot 框架、JPA 与 Hibernate、H2 数据库、前端技术以及 Maven 构建工具等，为后续章节的系统设计与实现提供技术基础。

2.1 Spring Boot 框架

Spring Boot 是由 Pivotal 团队开发的基于 Spring 框架的快速应用开发框架，旨在简化 Spring 应用的初始搭建与开发过程。Spring Boot 秉承“约定优于配置”的设计理念，通过自动配置机制大幅减少了繁琐的 XML 配置工作，使开发者能够将更多精力聚焦于业务逻辑的实现[6]。

Spring Boot 的核心特性包括以下几个方面。第一，自动配置（Auto Configuration）：Spring Boot 根据项目中引入的依赖自动配置相关的 Bean 和组件，例如引入 spring-boot-starter-data-jpa 后，框架会自动配置 DataSource、EntityManagerFactory 等 Bean，无需手动编写配置代码。第二，内嵌 Web 容器：Spring Boot 内嵌了 Tomcat、Jetty 等 Servlet 容器，应用可以直接以 Java Application 的方式启动，无需部署到外部容器，极大简化了开发与部署流程。第三，Starter 依赖管理：Spring Boot 提供了一系列 starter 依赖包，每个 starter 聚合了特定功能所需的全部依赖，开发者只需引入一个 starter 即可获得完整的功能支持，有效避免了依赖冲突问题。第四，生产级监控：Spring Boot Actuator 提供了健康检查、运行指标监控等开箱即用的运维功能[7]。

本系统采用 Spring Boot 4.1.0 版本，利用其自动配置特性快速搭建项目骨架，通过 spring-boot-starter-webmvc 提供 RESTful API 能力，通过 spring-boot-starter-data-jpa 简化持久层开发，通过 spring-boot-starter-validation 实现参数校验。Spring Boot 的 Starter 依赖管理机制确保了各组件版本的兼容性，Maven 插件支持一键打包生成可执行 JAR 文件，显著提升了开发效率。

在配置管理方面，Spring Boot 采用 application.properties 文件作为统一配置入口，支持以键值对形式配置数据库连接、服务端口、日志级别等参数。本系统在 application.properties 中配置了 H2 数据库连接字符串（jdbc:h2:file:./data/tailordb）、JPA 的 ddl-auto 策略（update，自动更新表结构）、H2 控制台启用

（spring.h2.console.enabled=true）等关键参数。Spring Boot 的配置覆盖机制还支持通过命令行参数、环境变量和 profile 配置文件灵活调整运行参数，满足不同环境的部

署需求。此外，Spring Boot 4.1.0 对 Java 25 提供了良好的兼容支持，能够利用最新 Java 版本的性能优化和语言特性。

2.2 JPA 与 Hibernate

JPA（Java Persistence API）是 Java 平台标准的对象关系映射（ORM）规范，定义了一套将 Java 对象映射到关系数据库表的 API 和元数据。JPA 的核心思想是通过注解或 XML 描述对象与数据库表之间的映射关系，使开发者能够以面向对象的方式操作数据库，而无需编写大量的 JDBC 模板代码[8]。

Hibernate 是 JPA 规范最流行的实现框架之一，提供了完整的 ORM 解决方案。Hibernate 支持丰富的映射注解，包括@Entity（标识实体类）、@Table（指定表名）、@Column（映射列属性）、@Id（主键标识）、@GeneratedValue（主键生成策略）、@ManyToOne（多对一关系）等。通过这些注解，开发者可以精确地描述实体类与数据库表之间的映射关系，包括字段类型、长度约束、外键关联等。Hibernate 还提供了 JPQL（Java Persistence Query Language）查询语言，支持面向对象的查询语法，以及 Criteria API 等动态查询机制，满足复杂业务场景的查询需求[9]。

Spring Data JPA 在 Hibernate 的基础上进一步简化了数据访问层的开发。开发者只需定义一个继承 JpaRepository 的接口，Spring Data JPA 会根据方法名自动生成查询实现，例如 findByIdOrderByCreateTimeDesc 方法会自动生成按用户 ID 查询并按创建时间倒序排列的 SQL 语句。同时，Spring Data JPA 还支持@Query 注解自定义 JPQL 查询，以及基于方法名的派生查询，极大减少了数据访问层的样板代码。本系统使用 Spring Data JPA 定义了 AdminUserRepository、UserRepository、FabricRepository、StyleRepository、MeasurementRepository 和 OrderRepository 共 6 个 Repository 接口，实现了各实体的 CRUD 操作与自定义查询。

在本系统的实体设计中，JPA 注解发挥了关键作用。例如，Fabric 实体的 description 字段使用@Column(length = 1000)注解将字段长度扩展至 1000 个字符，避免了默认 VARCHAR(255)长度对面料描述内容的截断；Order 实体通过@ManyToOne 注解建立与 User、Style、Fabric 和 Measurement 的多对一关联关系，并通过外键约束保障引用完整性；Measurement 实体的 remark 字段使用@Column(length = 500)注解以容纳详细的量体备注信息。此外，所有实体均使用@GeneratedValue(strategy = GenerationType.IDENTITY)注解配置主键自增策略，@Table 注解指定数据库表名，确保实体与表的精确映射。Hibernate 的 ddl-

auto=update 策略使得应用启动时自动根据实体类定义创建或更新数据库表结构，开发期间无需手动执行 DDL 语句。

2.3 H2 数据库

H2 数据库是一个用纯 Java 编写的开源关系型数据库管理系统，具有体积小、速度快、零配置等特点。H2 支持多种运行模式，包括嵌入式模式（Embedded Mode）、服务器模式（Server Mode）和混合模式。在嵌入式模式下，H2 数据库与应用程序运行在同一 JVM 中，无需单独安装和配置数据库服务，非常适合开发和测试环境使用[10]。

H2 数据库支持标准 SQL 语法，兼容 ANSI SQL-92 标准，提供了对视图、触发器、存储过程、外键约束等关系型数据库核心特性的完整支持。H2 还内置了 Web Console 管理界面，开发者可通过浏览器访问 H2 Console 查看和操作数据库表结构与数据，极大方便了开发调试。在本系统中，H2 Console 通过 `spring.h2.console.enabled=true` 配置启用，访问路径为/h2-console。

本系统选择 H2 数据库的文件模式（`jdbc:h2:file:./data/tailordb`）进行数据持久化。文件模式下，H2 将数据存储在本地的文件系统的 `.mv.db` 文件中，应用重启后数据不会丢失，实现了真正的数据持久化。相较于内存模式（`jdbc:h2:mem:`），文件模式更适合需要数据累积的业务场景。H2 数据库与 JPA/Hibernate 的集成非常简便，只需在 `application.properties` 中配置 `spring.datasource.url=jdbc:h2:file:./data/tailordb` 即可，Hibernate 的 `ddl-auto=update` 策略会根据实体类定义自动创建和更新表结构。此外，H2 数据库对 JPA 的关联查询、分页查询等特性提供了良好的支持，满足了本系统的数据访问需求。

本系统采用 H2 数据库的文件模式进行数据持久化，配置连接字符串为 `jdbc:h2:file:./data/tailordb`，数据文件存储在项目的 `data` 目录下，应用程序重启后数据不会丢失。H2 数据库兼容 MySQL 语法模式（`MODE=MySQL`），便于未来迁移至 MySQL 等生产级数据库。此外，系统启用了 H2 Web 控制台

（`spring.h2.console.enabled=true`），开发者可通过浏览器访问/h2-console 路径，直观地查看和操作数据库中的表结构与数据，为开发调试提供了便利。H2 数据库的轻量级特性使得本系统无需额外安装数据库服务，降低了部署门槛，同时其 ACID 事务支持保障了数据操作的一致性与可靠性。

2.4 HTML/CSS/JavaScript

HTML（HyperText Markup Language）是构建 Web 页面的标准标记语言，通过语义化标签描述网页的结构与内容。本系统前端采用 HTML5 标准，使用 `header`、`nav`、`section`、`article` 等语义化标签构建页面结构，提升了页面的可读性和可维护性，同时有利于搜索引擎优化和无障碍访问[11]。

CSS（Cascading Style Sheets）负责网页的样式与布局控制。本系统使用 CSS3 进行样式设计，采用 Flexbox 弹性布局实现响应式卡片排列，通过 CSS 变量统一管理主题色彩，利用过渡动画（`transition`）增强用户交互体验。系统的面料展示、款式展示等页面采用卡片式布局，每张卡片包含图片、名称、价格等关键信息，视觉层次清晰。

JavaScript 是实现网页动态交互的核心脚本语言。本系统前端使用原生 JavaScript（Vanilla JS）开发，通过 `fetch` API 与后端 RESTful 接口进行异步通信，利用 DOM 操作实现页面内容的动态渲染与更新。JavaScript 的事件驱动机制用于处理用户点击、表单提交等交互行为，例如面料筛选、款式搜索、订单创建等操作均通过 JavaScript 向后端发送请求并更新页面内容。相较于 Vue、React 等前端框架，原生 JavaScript 方案无需构建工具和额外依赖，部署简单，适合本课程设计项目的规模与需求[12]。

在异步通信方面，本系统前端统一使用 `fetch` API 替代传统的 `XMLHttpRequest` 对象。`fetch` API 基于 Promise 设计，支持 `async/await` 语法，使异步代码的可读性大幅提升。例如，用户登录功能通过 `fetch` 向 `/api/users/login` 端点发送 POST 请求，`await` 等待响应后根据 HTTP 状态码判断登录是否成功，成功则更新页面显示用户信息，失败则显示错误提示。这种基于 Promise 的异步模式避免了回调地狱（Callback Hell）问题，使代码结构更加清晰。此外，所有 `fetch` 请求均统一处理网络异常和服务器错误，通过 `try-catch` 捕获异常并向用户展示友好的错误信息。

在 DOM 操作方面，系统使用 `document.querySelector` 和 `document.querySelectorAll` 选择器获取页面元素，通过 `innerHTML`、`textContent` 和 `classList` 等 API 动态更新页面内容和样式。面料和款式的卡片展示通过 JavaScript 动态生成 HTML 字符串并插入到容器元素中，实现了数据驱动的 UI 渲染。订单进度条通过 CSS 类名的动态添加与移除实现节点高亮效果，使用户能够直观地查看订单当前所处的生产阶段。模态框的显示与隐藏通过修改元素的 `display` 样式属性实现，并支持点击遮罩层关闭模态框的交互模式。

2.5 Maven 构建工具

Maven 是 Apache 软件基金会开发的项目管理与构建自动化工具，基于项目对象模型（POM，Project Object Model）理念。Maven 通过 pom.xml 配置文件统一管理项目的依赖库、构建流程和项目元信息，实现了项目构建的标准化与自动化[13]。

Maven 的核心概念包括 POM 文件、坐标系统、依赖管理和生命周期。POM 文件是 Maven 项目的核心配置文件，定义了项目的基本信息、依赖列表、插件配置和构建规则。坐标系统通过 groupId、artifactId 和 version 三个元素唯一标识一个项目或依赖库，确保依赖的准确解析。Maven 的依赖管理机制支持传递依赖解析和版本冲突调解，开发者只需声明直接依赖，Maven 会自动解析并下载所有间接依赖。

Maven 定义了 clean、validate、compile、test、package、verify、install、deploy 等标准生命周期阶段，通过命令行即可执行完整的构建流程[14]。

本系统使用 Maven 进行项目构建与依赖管理，pom.xml 文件中声明了 spring-boot-starter-webmvc、spring-boot-starter-data-jpa、spring-boot-starter-validation、spring-boot-h2console、h2、spring-security-crypto 等核心依赖。同时，项目集成了 Maven Wrapper（mvnw），使开发者无需预装 Maven 即可使用项目内置的 Maven 版本进行构建，保证了构建环境的一致性。通过 spring-boot-maven-plugin 插件，项目支持一键打包生成可执行 JAR 文件，简化了部署流程[15]。

第3章 需求分析

需求分析是软件开发生命周期中的关键阶段，其质量直接影响系统设计的合理性与最终交付质量。本章从功能性需求和非功能性需求两个维度对服装私人定制网站系统进行需求分析，为后续的系统设计与实现提供依据[16]。

3.1 功能性需求分析

功能性需求描述系统应当具备的具体功能和行为。根据服装私人定制业务流程的分析，本系统的功能性需求分为前台用户功能和后台管理功能两大类[17]。

3.1.1 前台用户功能需求

前台用户功能面向普通消费者，主要包括以下功能模块：

（1）用户注册与登录：用户通过用户名和密码进行注册，注册时密码采用BCrypt加密存储。登录成功后系统创建Session保存用户信息，后续请求通过Session进行身份认证。用户可修改登录密码。

（2）面料浏览与筛选：用户可浏览所有上架面料，支持按材质、颜色、价格范围进行筛选，面料信息包括名称、材质、颜色、单价、库存量和描述。

（3）款式浏览与搜索：用户可浏览所有上架款式，支持按类别筛选和关键词搜索，款式信息包括名称、类别、工费和描述。

（4）量体数据管理：用户可录入、编辑和删除自己的量体数据，量体数据包含身高、体重、颈围、肩宽、胸围、腰围、臀围、衣长、袖长、裤长、大腿围共14项身体尺寸及备注信息。系统对量体数据进行归属校验，防止用户访问或修改他人的量体数据。

（5）订单创建：用户选择款式、面料和量体数据后创建定制订单，系统自动计算订单总价（总价 = 款式工费 + 面料单价 × 3 米），订单初始状态为待确认（PENDING）。

（6）订单查询：用户可查看自己的所有订单信息，包括订单编号、所选款式与面料、量体数据、总价、当前状态和下单时间，并可查看订单进度。

3.1.2 后台管理功能需求

后台管理功能面向系统管理员，主要包括以下功能模块：

（1）管理员登录：管理员通过独立的登录入口进行认证，与前台用户分表存储，Session 中使用不同的键进行区分。

（2）仪表盘统计：管理员登录后可查看系统概览数据，包括用户总数、订单总数、面料总数和款式总数等统计信息。

（3）面料管理：管理员可添加、编辑和删除面料信息，管理面料的名称、材质、颜色、单价、库存和描述。

（4）款式管理：管理员可添加、编辑和删除款式信息，管理款式的名称、类别、工费和描述。

（5）订单管理：管理员可查看全部订单，并推进订单状态（待确认→量体中→裁剪中→缝制中→试衣中→已完成），状态推进为单向操作，不允许跳跃或回退。

（6）用户管理：管理员可查看所有注册用户的基本信息。

3.1.3 系统用例分析

从用户角色角度分析，系统涉及两类主要参与者：前台消费者（User）和后台管理员（AdminUser）。前台消费者的核心用例包括注册账号、登录系统、浏览面料、筛选面料、浏览款式、搜索款式、录入量体数据、编辑量体数据、删除量体数据、创建订单和查看订单进度。后台管理员的核心用例包括登录后台、查看仪表盘统计、添加面料、编辑面料、删除面料、添加款式、编辑款式、删除款式、查看全部订单、推进订单状态和查看用户列表。

从业务流程角度分析，系统的核心业务流程为“定制下单流程”，该流程贯穿前台用户和后台管理员两个角色。前台用户完成注册登录、量体数据录入、面料和款式选择、订单创建等前置环节；后台管理员接收订单后，依次推进订单状态（待确认→量体中→裁剪中→缝制中→试衣中→已完成），完成定制服装的生产交付。整个流程通过订单状态机进行管理，确保每个环节按序执行，不允许跳跃或回退。

此外，系统还存在一些约束性用例。例如，用户修改密码需要验证旧密码；量体数据的编辑和删除需要校验数据归属；管理员推进订单状态需要检查当前状态是否为终态。这些约束性用例保障了系统操作的安全性和数据的一致性。

表 3-1 功能性需求汇总表

功能模块	功能项	操作角色	功能描述
用户管理	注册	访客	用户名+密码注册，BCrypt 加密存储
用户管理	登录	访客	用户名+密码登录，Session 认证
用户管理	修改密码	前台用户	验证旧密码后设置新密码
面料管理	浏览筛选	前台用户	按材质/颜色/价格筛选面料

功能模块	功能项	操作角色	功能描述
面料管理	增删改查	管理员	面料信息全量管理
款式管理	浏览搜索	前台用户	按类别/关键词搜索款式
款式管理	增删改查	管理员	款式信息全量管理
量体数据	录入编辑	前台用户	14 项尺寸录入，归属校验
订单管理	下单	前台用户	选款式+面料+量体→计算总价
订单管理	状态推进	管理员	PENDING→...→COMPLETED 单向推进
后台管理	仪表盘	管理员	用户/订单/面料/款式统计

3.2 非功能性需求分析

非功能性需求是对系统运行质量与约束条件的描述，直接影响用户体验和系统可靠性。本系统的非功能性需求主要包括以下几个方面[18]：

3.2.1 安全性需求

（1）密码安全：用户密码不得以明文形式存储，必须采用 BCrypt 加密算法进行加密处理，BCrypt 算法内置盐值机制，有效防止彩虹表攻击。

（2）身份认证：系统采用基于 Session 的认证机制，用户登录成功后在服务端创建 Session 并保存用户信息，后续请求通过 Session ID 进行身份识别。

（3）权限控制：通过拦截器实现 URL 级别的权限控制，GET 请求（浏览查询）允许匿名访问，POST/PUT/DELETE 请求必须经过身份认证；后台管理接口（/api/admin/**）仅允许管理员访问；前台用户写操作（量体、订单）必须前台用户登录。

（4）数据隔离：量体数据和订单数据实行用户级数据隔离，用户只能访问和操作属于自己的数据，系统在 Service 层进行数据归属校验，防止越权访问。

3.2.2 性能需求

（1）响应速度：系统页面加载时间应控制在 3 秒以内，API 接口响应时间应控制在 500 毫秒以内。H2 数据库文件模式提供本地高速读写能力，JPA 的一级缓存机制减少不必要的数据库查询。

（2）并发处理：系统应支持多用户同时访问，Spring Boot 内嵌 Tomcat 容器支持多线程并发请求处理。

3.2.3 可用性需求

（1）界面友好：前端界面采用响应式设计，适配不同屏幕尺寸；操作流程简洁直观，关键操作提供明确的成功或错误提示。

（2）错误处理：后端对非法参数、资源不存在、权限不足等异常进行统一处理，返回规范的 HTTP 状态码和错误信息；前端对网络异常和业务错误进行友好提示。

3.2.4 可维护性需求

（1）分层架构：系统采用表现层、控制层、业务层、数据访问层的分层架构，各层职责清晰，降低耦合度，便于独立开发和维护。

（2）DTO 分离：系统使用 DTO（Data Transfer Object）进行数据传输，将实体对象与接口返回数据解耦，避免实体对象直接暴露给前端，同时便于接口数据的灵活组合。

（3）代码规范：遵循 Java 编码规范，类和方法均有完整的中文注释，包括参数说明、返回值说明和异常说明，核心业务逻辑添加行注释解释设计意图。

表 3-2 非功能性需求汇总表

需求类别	需求指标	实现方案
安全性	密码加密存储	BCryptPasswordEncoder 加密
安全性	登录态管理	HttpSession 服务端会话
安全性	接口权限控制	LoginInterceptor 拦截器
安全性	数据隔离	Service 层归属校验
性能	页面加载时间	3 秒以内
性能	API 响应时间	500 毫秒以内
性能	并发支持	Tomcat 多线程处理
可用性	界面适配	响应式 CSS 布局
可用性	错误处理	统一 HTTP 状态码
可维护性	架构分层	五层解耦架构
可维护性	数据传输	DTO 模式隔离
可维护性	代码注释	全中文注释

第4章 系统设计

本章在需求分析的基础上，对服装私人定制网站系统进行详细设计，包括设计原则、系统架构设计、功能模块设计和数据库设计四个方面，为后续的功能实现提供蓝图。

4.1 设计原则

系统设计遵循以下核心原则：

（1）分层解耦：系统采用经典的分层架构，将表现层、控制层、业务层和数据访问层进行分离，各层通过接口进行通信，降低层间耦合度，提高系统的可维护性和可测试性。

（2）单一职责：每个类和模块只负责一项功能职责，**Controller** 负责请求接收与响应封装，**Service** 负责业务逻辑处理，**Repository** 负责数据访问，**Model** 负责数据建模，**DTO** 负责数据传输，各司其职，避免“上帝类”问题。

（3）安全优先：在系统设计的各个环节贯彻安全理念，密码加密存储、**Session** 认证、拦截器鉴权、数据归属校验等多层安全机制协同工作，保障系统和用户数据安全。

（4）幂等设计：对于状态推进等关键操作，采用幂等设计理念，订单状态推进操作在已完成状态下重复调用不会产生副作用，保证系统在异常重试场景下的数据一致性。

4.2 系统架构设计

本系统采用 B/S（**Browser/Server**）架构，用户通过浏览器访问系统，前端页面通过 HTTP 协议与后端 **Spring Boot** 应用通信，后端通过 **JPA** 与 **H2** 数据库交互。系统整体架构分为五层，自上向下依次为表现层、控制层、业务层、数据访问层和数据层。

图4-1 系统架构图

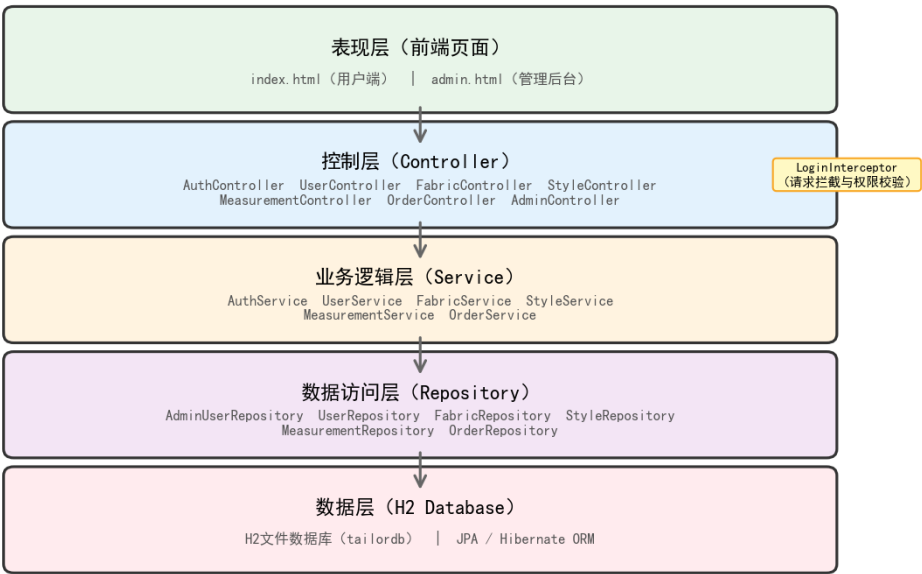


图 4-1 系统架构图

如图 4-1 所示，系统五层架构的职责划分如下：

（1）表现层：由 HTML/CSS/JavaScript 组成，负责页面展示与用户交互，通过 fetch API 向后端发送 RESTful 请求，接收 JSON 格式响应并动态渲染页面内容。

（2）控制层：由 Spring MVC 的 Controller 组件构成，负责接收 HTTP 请求、参数校验、调用业务层服务并封装响应数据。系统包含 AuthController、UserController、FabricController、StyleController、MeasurementController、OrderController 和 AdminController 共 7 个控制器。

（3）业务层：由 Service 组件构成，负责核心业务逻辑处理，包括密码加密验证、订单总价计算、量体数据归属校验、订单状态推进等。系统包含 AuthService、UserService、FabricService、StyleService、MeasurementService 和 OrderService 共 6 个服务类。

（4）数据访问层：由 Spring Data JPA 的 Repository 接口构成，负责数据库 CRUD 操作与自定义查询。系统包含 6 个 Repository 接口，通过方法名派生查询和 @Query 注解实现数据访问。

（5）数据层：由 H2 文件数据库构成，负责数据的持久化存储，包含管理员表、用户表、面料表、款式表、量体数据表和订单表共 6 张数据表。

各层之间通过接口进行通信，上层依赖下层接口而非实现，符合依赖倒置原则。表现层通过 `fetch API` 向控制层发送 `HTTP` 请求，控制层将请求转发至业务逻辑层处理，业务逻辑层调用数据访问层完成数据持久化操作。`LoginInterceptor` 作为横切关注点，在请求到达控制层之前进行统一的权限校验，拦截未授权的访问请求。`DataInitializer` 在应用启动时执行幂等的数据初始化操作，确保系统首次启动时具备完整的演示数据。此外，系统采用 `DTO` 模式进行数据传输隔离，请求 `DTO` 使用 `Jakarta Validation` 注解进行参数校验，响应 `DTO` 过滤了密码等敏感字段，并根据前端展示需求组装关联实体的名称信息，避免了 `N+1` 查询问题。

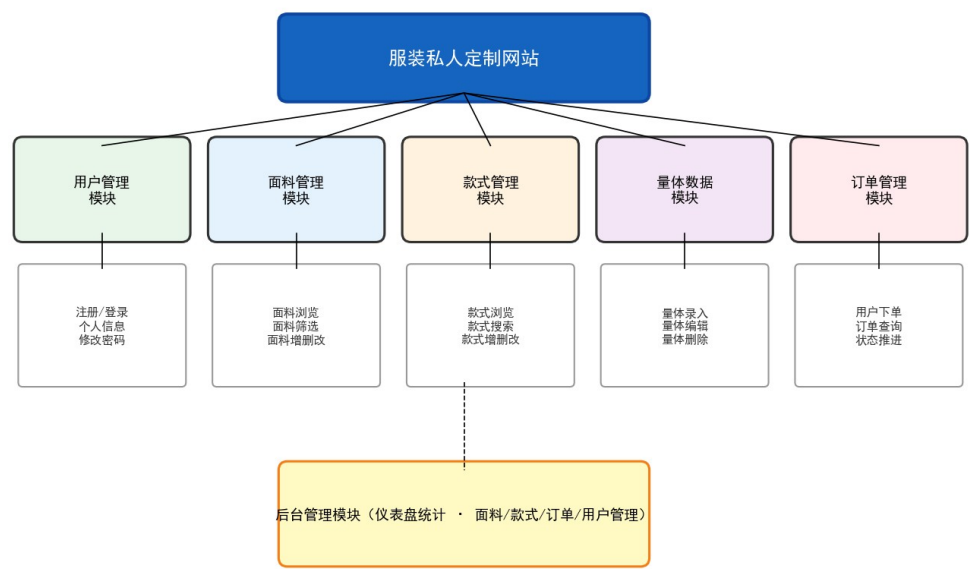
表 4-1 技术选型表

技术领域	技术/工具	版本	选型理由
后端框架	Spring Boot	4.1.0	自动配置、内嵌容器、 <code>Starter</code> 依赖
开发语言	Java	25	最新 LTS 版本，语法特性丰富
ORM 框架	Spring Data JPA	随 Boot	简化持久层开发，方法名派生查询
数据库	H2 Database	随 Boot	纯 Java、文件模式、零安装
前端技术	HTML/CSS/JS	HTML5	原生技术栈，无需构建工具
构建工具	Maven	3.9+	标准化构建、依赖管理
密码加密	BCrypt	随 Security	内置盐值、抗彩虹表攻击
认证机制	Session	Servlet	服务端会话管理，简单可靠

4.3 功能模块设计

根据需求分析，系统功能划分为用户管理、面料管理、款式管理、量体数据管理、订单管理和后台管理六大模块。各功能模块之间的关系如图 4-2 所示。

图4-2 系统功能模块图



如图 4-2 所示，系统的六大功能模块相互协作，共同支撑服装定制的完整业务流程。前台用户通过用户管理模块完成注册登录后，可使用面料管理、款式管理和量体数据管理模块浏览和录入定制所需信息，进而通过订单管理模块创建定制订单。后台管理模块为管理员提供系统全局管理能力，包括数据统计、信息维护和订单状态推进等功能。

表 4-2 功能模块设计表

模块名称	子功能	涉及实体	关键接口
用户管理	注册/登录/改密	User, AdminUser	POST /api/users/register
面料管理	浏览/筛选/增删改	Fabric	GET /api/fabrics
款式管理	浏览/搜索/增删改	Style	GET /api/styles
量体数据	录入/编辑/删除	Measurement	POST /api/measurements
订单管理	下单/查询/推进	Order	POST /api/orders
后台管理	仪表盘/全量管理	全部实体	GET /api/admin/stats

在用户管理模块设计中，系统采用了前台用户与管理员分表存储的策略。AdminUser 表和 User 表分别独立存储管理员和前台用户的数据，两表之间不存在外键关联。这种设计实现了前台与后台权限的物理隔离，即使前台用户数据被泄露也无法影响后台管理系统的安全。在 Session 管理方面，系统使用不同的 Session Key（adminUser 和 frontUser）区分两种用户角色的登录状态，LoginInterceptor 根据请求路径和 Session Key 进行差异化权限校验。

在面料管理模块设计中，Fabric 实体的 description 字段使用@Column(length = 1000)注解将字段长度扩展至 1000 个字符，以容纳详细的面料描述信息。面料筛选功能支持按材质（material）、颜色（color）和最大价格（maxPrice）三个维度进行组合筛选，筛选逻辑在 FabricService 中通过 Java Stream API 实现内存过滤。款式管理模块的设计与面料管理类似，Style 实体的 description 字段同样使用@Column(length = 1000)注解，筛选功能支持按类别（category）和关键词（keyword）进行搜索。

在量体数据管理模块设计中，Measurement 实体包含 14 项身体尺寸字段，涵盖了上衣和裤装定制所需的全部关键尺寸。量体数据的增删改操作均在 Service 层进行归属校验，通过比较 measurement.getUser().getId()与当前登录用户的 ID 来判断操作权限。这种数据级别的权限控制有效防止了水平越权漏洞，即用户只能操作自己的量体数据。

在订单管理模块设计中，Order 实体通过四个@ManyToOne 关联（User、Style、Fabric、Measurement）建立了与其它实体的多对一关系。订单创建时，系统自动计算总价（style.craftFee + fabric.unitPrice * 3.0），并设置初始状态为 PENDING。订单状态推进通过 OrderStatus 枚举的 ordinal 值实现，每次推进仅允许前进一格，COMPLETED 为终态不可变更。这种基于枚举序号的状态机设计简洁可靠，保证了状态流转的单向性。

4.4 数据库设计

4.4.1 E-R 图设计

根据系统功能需求，数据库共设计 6 张实体表，分别为管理员表（AdminUser）、前台用户表（User）、面料表（Fabric）、款式表（Style）、量体数据表（Measurement）和订单表（Order）。各实体之间的 E-R 关系如图 4-3 所示。

图4-3 数据库E-R图

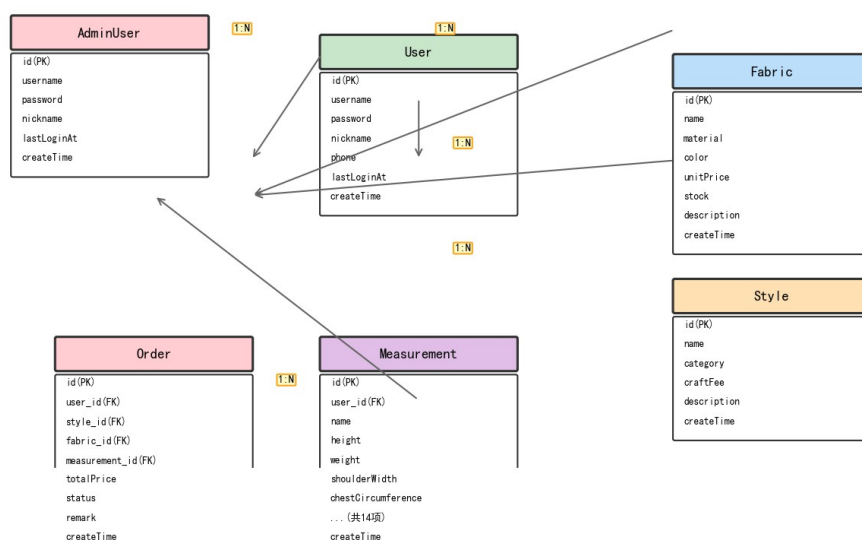


图 4-3 E-R 图

如图 4-3 所示，系统实体间的关系如下：一个前台用户可以拥有多条量体数据（User 1:N Measurement），一个前台用户可以创建多个订单（User 1:N Order），一个款式可以被多个订单引用（Style 1:N Order），一个面料可以被多个订单引用（Fabric 1:N Order），一条量体数据可以关联多个订单（Measurement 1:N Order）。管理员表与前台用户表独立存储，互不关联。

在数据库设计过程中，遵循了关系型数据库的规范化原则。各实体表均满足第三范式（3NF），即每个非主属性既不部分依赖于候选码也不传递依赖于候选码。例如，订单表中不存储面料名称和款式名称等冗余信息，而是通过外键引用 Fabric 和 Style 实体的主键，在查询时通过 JPA 的关联查询获取关联数据。这种设计避免了数据冗余和更新异常，保障了数据的一致性。

在外键约束设计方面，Order 表通过 user_id、style_id、fabric_id 和 measurement_id 四个外键分别关联 User、Style、Fabric 和 Measurement 表的主键。JPA 通过 @ManyToOne 注解自动创建外键约束，确保引用完整性。当删除 User、Style 或 Fabric 记录时，由于存在外键约束引用，需要先处理关联的 Order 记录。Measurement 表通过 user_id 外键关联 User 表，表示量体数据的归属关系，这是实现数据隔离的数据库层面基础。

4.4.2 订单状态流转设计

订单状态流转是系统核心业务逻辑之一，订单从创建到完成需经过 6 个状态阶段，状态推进为单向操作，不允许跳跃或回退。订单状态流转如图 4-4 所示。

图4-4 订单状态流转图

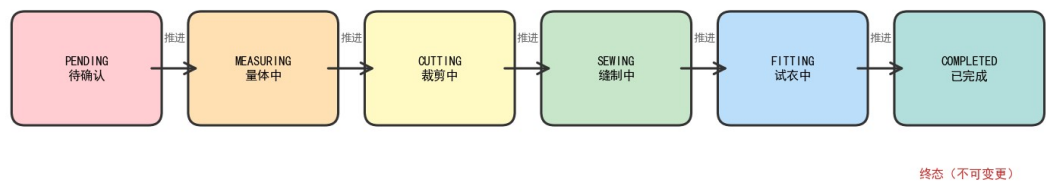


图 4-4 订单状态流转图

如图 4-4 所示，订单状态的 6 个阶段及其含义如下：

- （1）PENDING（待确认）：用户刚下单，等待管理员确认订单信息。
- （2）MEASURING（量体中）：管理员确认后进入量体环节，即使已自助录入尺寸，仍需工坊二次复核。
- （3）CUTTING（裁剪中）：依据量体数据裁剪面料。
- （4）SEWING（缝制中）：进入工坊缝制成衣。
- （5）FITTING（试衣中）：成衣完成后通知客户试穿。
- （6）COMPLETED（已完成）：客户确认合身，订单流程结束。

状态推进逻辑在 OrderService 的 advanceStatus 方法中实现，通过 OrderStatus 枚举的 ordinal()值计算下一个状态索引，确保状态只能按顺序前进一格。当订单状态已为 COMPLETED 时，后续推进操作将被拒绝，保证流程的终态稳定性。

4.4.3 数据表结构设计

根据 E-R 图设计，系统共创建 6 张数据表，各表结构如下。

表 4-3 管理员表（admin_user）结构

字段名	类型	约束	说明
id	BIGINT	PK, 自增	主键 ID
username	VARCHAR	NOT NULL, UNIQUE	管理员用户名
password	VARCHAR	NOT NULL	BCrypt 加密密码
nickname	VARCHAR	—	管理员昵称
last_login_at	TIMESTAMP	—	最后登录时间

字段名	类型	约束	说明
create_time	TIMESTAMP	—	创建时间

表 4-4 前台用户表（user）结构

字段名	类型	约束	说明
id	BIGINT	PK, 自增	主键 ID
username	VARCHAR	NOT NULL, UNIQUE	用户名
password	VARCHAR	NOT NULL	BCrypt 加密密码
nickname	VARCHAR	—	用户昵称
phone	VARCHAR	—	联系电话
last_login_at	TIMESTAMP	—	最后登录时间
create_time	TIMESTAMP	—	注册时间

表 4-5 面料表（fabric）结构

字段名	类型	约束	说明
id	BIGINT	PK, 自增	主键 ID
name	VARCHAR	NOT NULL	面料名称
material	VARCHAR	—	材质（如棉、麻、丝）
color	VARCHAR	—	颜色
unit_price	DOUBLE	NOT NULL	单价（元/米）
stock	INT	—	库存量
description	VARCHAR	—	面料描述
create_time	TIMESTAMP	—	创建时间

表 4-6 款式表（style）结构

字段名	类型	约束	说明
id	BIGINT	PK, 自增	主键 ID
name	VARCHAR	NOT NULL	款式名称
category	VARCHAR	—	类别（如西装、衬衫）
craft_fee	DOUBLE	NOT NULL	工费（元）
description	VARCHAR	—	款式描述
create_time	TIMESTAMP	—	创建时间

表 4-7 量体数据表（measurement）结构

字段名	类型	约束	说明
id	BIGINT	PK, 自增	主键 ID
user_id	BIGINT	FK→user.id	所属用户 ID

字段名	类型	约束	说明
name	VARCHAR	—	量体记录名称
height	DOUBLE	—	身高
weight	DOUBLE	—	体重
neck_circumference	DOUBLE	—	颈围
shoulder_width	DOUBLE	—	肩宽
chest_circumference	DOUBLE	—	胸围
waist_circumference	DOUBLE	—	腰围
hip_circumference	DOUBLE	—	臀围
clothes_length	DOUBLE	—	衣长
sleeve_length	DOUBLE	—	袖长
pants_length	DOUBLE	—	裤长
thigh_circumference	DOUBLE	—	大腿围
remark	VARCHAR	—	备注
create_time	TIMESTAM P	—	创建时间

表 4-8 订单表（orders）结构

字段名	类型	约束	说明
id	BIGINT	PK, 自增	主键 ID
user_id	BIGINT	FK→user.id	下单用户 ID
style_id	BIGINT	FK→style.id	所选款式 ID
fabric_id	BIGINT	FK→fabric.id	所选面料 ID
measurement_id	BIGINT	FK→measurement.id	量体数据 ID
total_price	DOUBLE	NOT NULL	订单总价（元）
status	VARCHAR	NOT NULL	订单状态枚举
remark	VARCHAR	—	客户备注
create_time	TIMESTAM P	—	下单时间

上述 6 张数据表的设计充分考虑了服装定制业务的数据管理需求。管理员表和前台用户表采用了相似的字段结构但分表存储，实现了权限的物理隔离。面料表和款式表通过 `description` 字段的长度扩展（1000 字符）支持详细的产品描述。量体数据表包含了 14 项身体尺寸字段，覆盖了颈围、肩宽、胸围、腰围、臀围、衣长、袖长、裤长、大腿围等定制服装所需的关键测量点，每项尺寸均使用 `DOUBLE` 类型存储以保留小数精度。订单表通过四个外键字段（`user_id`、`style_id`、`fabric_id`、

measurement_id）建立了与其它实体的关联关系，status 字段存储订单状态枚举的字符串值。

在字段类型选择方面，主键统一使用 BIGINT 自增类型，保证主键的唯一性和递增性。密码字段使用 VARCHAR 类型存储 BCrypt 加密后的 60 位哈希字符串。价格和尺寸类数值字段使用 DOUBLE 类型，满足小数精度需求。时间字段使用 TIMESTAMP 类型，精确记录到秒级。文本类字段根据内容长度需求设置不同的 VARCHAR 长度：用户名和昵称使用默认 255 字符，描述类字段扩展至 500-1000 字符，备注字段设置为 500 字符。这种差异化的字段长度设计既避免了数据截断，又控制了存储空间的浪费。

4.4.3 订单状态流转设计

订单状态流转是系统核心业务逻辑之一，本系统采用枚举类型 OrderStatus 定义了 6 个订单状态，按序为 PENDING（待确认）、MEASURING（量体中）、CUTTING（裁剪中）、SEWING（缝制中）、FITTING（试衣中）和 COMPLETED（已完成）。状态流转规则为单向推进，每次推进仅允许前进一格，不允许跳跃或回退，COMPLETED 为终态不可变更。如图 4-4 所示。

图4-4 订单状态流转图

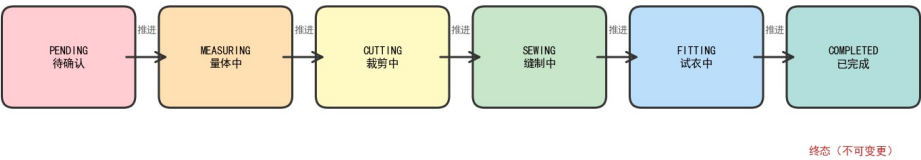


图 4-4 订单状态流转图

订单状态推进的实现逻辑位于 OrderService 的 advanceStatus 方法中。该方法首先查询订单是否存在，然后检查当前状态是否为 COMPLETED 终态（如果是则返回 null 表示不可推进），最后通过 OrderStatus.values() 数组获取当前状态的 ordinal 值，将状态设置为下一个枚举值。这种基于枚举序号的状态机设计简洁可靠，保证了状态流转的单向性和终态稳定性。管理员通过 PUT /api/admin/orders/{id}/advance 接口触发状态推进，系统返回更新后的订单信息或 404 错误（订单不存在或已完成）。

第5章 功能实现

本章详细描述服装私人定制网站系统各功能模块的实现过程，包括登录系统功能、信息展示功能、信息查询功能和信息改动功能四个方面，并结合关键代码和流程图说明实现细节。

5.1 登录系统功能

登录系统是系统的安全入口，负责用户身份认证与会话管理。系统将前台用户和管理员分表存储，前台用户使用 `User` 实体，管理员使用 `AdminUser` 实体，两者通过不同的 `Controller` 和 `Session` 键进行区分。

5.1.1 用户注册与登录

用户注册时，系统接收用户名和密码，使用 `BCryptPasswordEncoder` 对密码进行加密后存储至数据库。`BCrypt` 算法在每次加密时生成随机盐值并嵌入密文中，即使两个用户设置相同的密码，数据库中存储的密文也不同，有效防止彩虹表攻击。注册前系统会检查用户名是否已存在，确保用户名的唯一性。

用户登录时，系统根据用户名查询用户记录，使用 `BCryptPasswordEncoder` 的 `matches` 方法对用户输入的明文密码与数据库中的密文进行比对。验证通过后，将 `User` 对象存入 `HttpSession` 中，键名为 `SESSION_USER_KEY`。后续请求通过 `Session` 中的用户对象进行身份识别。用户修改密码时，需先验证旧密码正确性，再对新密码进行 `BCrypt` 加密后更新。

5.1.2 管理员登录

管理员登录流程与用户登录类似，但使用独立的 `AuthController` 和 `AdminUser` 实体。管理员登录成功后，将 `AdminUser` 对象存入 `Session`，键名为 `SESSION_ADMIN_KEY`。系统通过 `LoginInterceptor` 拦截器区分前台用户和管理员的访问权限。

5.1.3 拦截器权限控制

`LoginInterceptor` 实现了 `HandlerInterceptor` 接口，在 `preHandle` 方法中根据请求 `URI` 和 `HTTP` 方法进行权限判定。拦截规则如下：认证相关接口（登录、登出、注册）始终放行；`GET` 请求（浏览查询）放行；`/api/admin/**` 路径下的写操作需管理

员 Session；其余写操作需前台用户 Session。未通过认证的请求返回 HTTP 401 状态码。登录认证的完整时序如图 4-6 所示。

图4-6 用户登录认证时序图

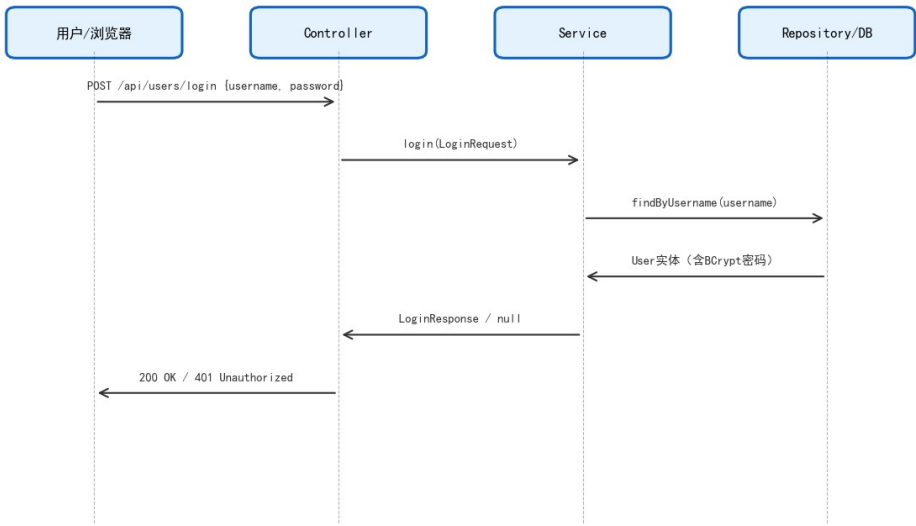


图 4-6 登录认证时序图

如图 4-6 所示，登录认证的完整流程为：用户在前端页面输入用户名和密码，通过 fetch API 发送 POST 请求至/api/users/login 接口；AuthController 接收请求并调用 AuthService 进行身份验证；AuthService 从 UserRepository 查询用户记录，使用 BCryptPasswordEncoder 比对密码；验证通过后将 User 对象存入 HttpSession，返回登录成功响应；前端接收响应后跳转至主页。后续请求携带 Session Cookie，LoginInterceptor 从 Session 中获取用户对象进行权限校验。

5.2 信息展示功能

信息展示功能负责向用户直观地展示面料、款式、订单和系统统计数据，是用户获取信息的主要途径。

5.2.1 面料卡片展示

系统前台首页以卡片形式展示所有面料信息，每张卡片包含面料名称、材质、颜色、单价和库存等关键字段。卡片采用 CSS Flexbox 布局，自动适应屏幕宽度，在不同设备上呈现良好的视觉效果。面料数据通过 GET /api/fabrics 接口获取，后端 FabricService 调用 FabricRepository 的 findAll 方法查询全部面料记录，并转换为 FabricResponse DTO 返回前端。

5.2.2 款式卡片展示

款式展示页面以与面料类似的卡片形式展示所有款式信息，每张卡片包含款式名称、类别、工费和描述。款式数据通过 GET /api/styles 接口获取，后端 StyleService 调用 StyleRepository 的 findAllOrderByCreateTimeDesc 方法查询全部款式并按创建时间倒序排列。

5.2.3 订单进度条可视化

用户订单页面以进度条形式可视化展示订单状态推进情况。进度条包含 6 个节点，分别对应 PENDING、MEASURING、CUTTING、SEWING、FITTING 和 COMPLETED 六个状态。前端根据订单当前状态的 ordinal 值，将已完成阶段的节点标记为高亮状态，使用户能够直观了解订单所处的生产阶段。订单数据通过 GET /api/orders 接口获取，后端 OrderService 调用 OrderRepository 的 findByIdOrderByCreateTimeDesc 方法查询当前用户的所有订单。

5.2.4 仪表盘统计卡片

后台管理首页的仪表盘以统计卡片形式展示系统概览数据，包括用户总数、订单总数、面料总数和款式总数。统计数据通过 GET /api/admin/stats 接口获取，后端 AdminController 调用各 Service 的 count 方法获取统计数据，封装为 StatsResponse DTO 返回。仪表盘使管理员能够快速了解系统运营概况。

5.3 信息查询功能

信息查询功能支持用户按条件筛选和搜索所需的面料、款式和订单信息，提升信息获取效率。

5.3.1 面料筛选

面料筛选支持按材质、颜色和价格范围进行多条件组合查询。前端提供筛选表单，用户可选择材质类型、颜色和价格区间，提交后通过 GET /api/fabrics 接口发送查询参数。后端 FabricService 根据传入的材质、颜色、最低价格和最高价格参数，调用 FabricRepository 的自定义查询方法进行条件过滤。当用户不指定筛选条件时，返回全部面料记录。

5.3.2 款式搜索

款式搜索支持按类别筛选和关键词模糊搜索。用户可选择款式类别（如西装、衬衫、外套等），也可在搜索框中输入关键词搜索款式名称。后端 `StyleService` 根据类别和关键词参数，调用 `StyleRepository` 的查询方法进行过滤。关键词搜索采用 `LIKE` 模糊匹配，查找款式名称中包含关键词的记录。

5.3.3 订单查询

订单查询功能分为前台用户查询和后台管理员查询两种模式。前台用户通过 `GET /api/orders` 接口查询自己的订单列表，后端 `OrderService` 通过 `Session` 获取当前登录用户 ID，调用 `OrderRepository` 的 `findByUserIdOrderByCreateTimeDesc` 方法查询该用户的所有订单，按下单时间倒序排列。后台管理员通过 `GET /api/admin/orders` 接口查询全部订单，后端调用 `findAllByOrderByCreateTimeDesc` 方法获取所有用户的订单记录。

5.4 信息改动功能

信息改动功能包括面料与款式的增删改、量体数据的管理和订单状态推进等写操作，均需通过身份认证和权限校验。

5.4.1 面料与款式增删改

管理员可通过后台管理界面对面料和款式信息进行增加、编辑和删除操作。面料管理通过 `POST /api/admin/fabrics`（新增）、`PUT /api/admin/fabrics/{id}`（修改）、`DELETE /api/admin/fabrics/{id}`（删除）三个接口实现。款式管理的接口结构与面料管理类似。后端 `Service` 层在执行修改和删除操作前，先通过 ID 查询记录是否存在，不存在则抛出 `IllegalArgumentException` 异常，前端接收异常信息并提示用户。

5.4.2 量体数据管理

量体数据管理是服装定制系统的核心功能之一，用户可录入 14 项身体尺寸数据。量体数据的增删改操作通过 `POST /api/measurements`（新增）、`PUT /api/measurements/{id}`（修改）、`DELETE /api/measurements/{id}`（删除）三个接口实现。

量体数据管理的关键安全设计是数据归属校验。在修改和删除操作中，后端 `MeasurementService` 首先通过 ID 查询量体数据记录，然后校验该记录的 `user_id` 是否与当前登录用户的 ID 一致。如果不一致，说明用户试图操作他人的量体数据，系统

抛出 `IllegalArgumentException` 异常拒绝操作。这一设计在 `Service` 层实现了数据级别的权限控制，有效防止了水平越权漏洞。用户下单的完整流程如图 4-5 所示。

图4-5 用户下单流程图

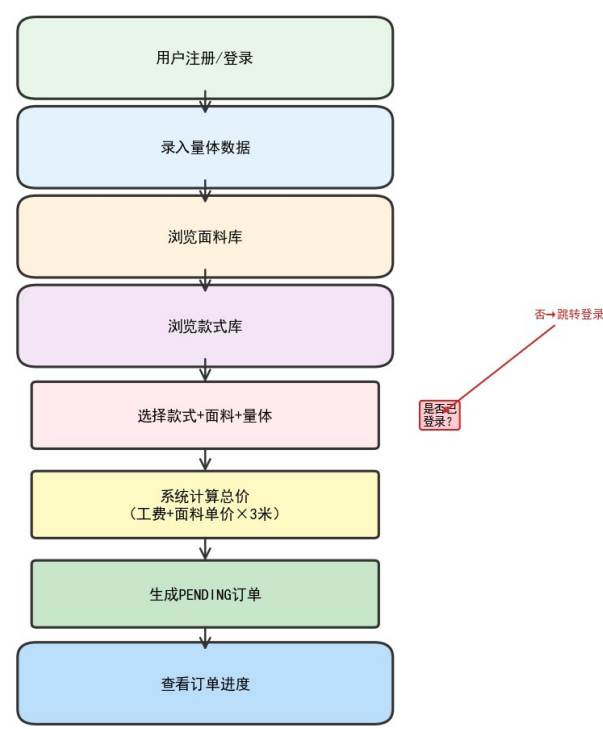


图 4-5 用户下单流程图

如图 4-5 所示，用户下单的完整流程为：用户登录后浏览面料列表，通过筛选功能选择合适的面料；浏览款式列表，通过搜索功能选择心仪的款式；进入量体数据管理页面，录入或选择已有的量体数据；在订单创建页面选择款式、面料和量体数据，系统自动计算订单总价（款式工费 + 面料单价 × 3 米）；用户确认信息后提交订单，系统创建订单记录并设置初始状态为 `PENDING`；用户可在订单页面查看订单详情和当前进度。

5.4.3 订单状态推进

订单状态推进由管理员在后台管理界面操作，通过 `PUT /api/admin/orders/{id}/advance` 接口实现。后端 `OrderService` 的 `advanceStatus` 方法首先通过 `ID` 查询订单记录，检查订单当前状态是否已为 `COMPLETED`，如果已完成则拒绝推进并返回 `null`。否则，通过 `OrderStatus` 枚举的 `values()` 方法获取状态数组，计算当前状态的 `ordinal` 值加 1 得到下一状态索引，更新订单状态并保存。状态推进为单向操作，确保生产流程按规范顺序执行。

订单总价计算是下单流程的核心逻辑，在 `OrderService` 的 `create` 方法中实现。计算公式为：总价 = 款式工费 (`style.getCraftFee()`) + 面料单价

(`fabric.getUnitPrice()`) × 默认用料量 (3.0 米)。其中，默认用料量定义为常量 `DEFAULT_FABRIC_USAGE = 3.0`。在创建订单前，系统还进行了多项校验：款式、面料和量体数据是否存在，以及量体数据是否属于当前登录用户。这些校验保障了订单数据的完整性和安全性。

在数据初始化方面，系统通过 `DataInitializer` 类实现 `CommandLineRunner` 接口，在应用启动时执行幂等的数据库初始化操作。`DataInitializer` 首先检查 `admin_users` 表是否为空，如果为空则执行初始化：创建 1 个默认管理员账号（`admin/admin123`，BCrypt 加密）、3 个示例用户（`alice`、`bob`、`carol`）、8 个示例面料（涵盖意大利羊毛、埃及长绒棉、日本亚麻、苏州真丝、蒙古羊绒、美国牛仔布、英国粗花呢、法国蕾丝等多种材质）、6 个示例款式（商务西装、法式衬衫、连衣裙、英伦风衣、西裤、修身旗袍）、4 组示例量体数据和 6 个覆盖不同状态的示例订单。初始化数据中订单的创建时间随机回拨 0 至 72 小时，模拟历史订单数据，使系统在首次启动时即具备完整的演示效果。

在 DTO 设计方面，系统采用了 Java 14+ 引入的 `record` 语法定义数据传输对象，共定义了 14 个 `record` 类。请求 DTO（如 `LoginRequest`、`RegisterRequest`、`FabricRequest` 等）使用 Jakarta Validation 注解（`@NotBlank`、`@NotNull`）进行参数校验，Controller 层通过 `@Valid` 注解触发自动校验。响应 DTO（如 `FabricResponse`、`OrderResponse` 等）不包含密码等敏感字段，`OrderResponse` 还包含了关联实体的名称信息（如款式名称、面料名称、用户昵称等），避免了前端需要额外发送请求获取关联数据的不便。`LoginResponse` 提供了 `from(AdminUser)` 和 `from(User, boolean)` 两个工厂方法，分别用于管理员和前台用户的登录响应构建。

在前端交互设计方面，`index.html` 用户端页面采用 Tab 导航栏组织五个主要功能区：首页、面料库、款式库、我的量体和我的订单。每个 Tab 对应一个独立的页面区块，通过 JavaScript 控制显示与隐藏。用户注册和登录功能以模态框形式呈现，无需跳转页面即可完成操作。面料和款式展示采用卡片网格布局，通过 CSS Flexbox 实现自适应排列。订单详情页面实现了 6 步进度条可视化，根据订单当前状态的 `ordinal` 值高亮已完成阶段的节点，使用户直观了解订单生产进度。

`admin.html` 管理后台页面采用侧边栏导航布局，包含仪表盘、面料管理、款式管理、订单管理和用户管理五个功能模块。管理员登录前显示独立的登录页面，登录成功后进入管理主界面。仪表盘页面以统计卡片形式展示系统概览数据（面料数、款式数、订单数、用户数），并展示最新 5 笔订单信息。面料管理和款式管理页面

采用表格形式展示数据，支持新增、编辑和删除操作，操作通过弹窗模态框完成。订单管理页面展示全部订单列表，每条订单记录提供详情查看和状态推进按钮，已完成订单不显示推进按钮。用户管理页面以表格形式展示所有注册用户的基本信息，包括注册时间和最近登录时间。

在异常处理与错误响应方面，系统采用了统一的异常处理策略。Controller 层使用 `@RestControllerAdvice` 注解的 `GlobalExceptionHandler` 全局异常处理器，捕获 `IllegalArgumentException`、`MethodArgumentNotValidException` 等异常并转换为规范的 JSON 错误响应。当用户提交的请求参数未通过 Jakarta Validation 校验时，系统返回 HTTP 400 状态码并在响应体中包含字段级别的错误信息。当用户未登录或权限不足时，`LoginInterceptor` 返回 HTTP 401 状态码。当请求的资源不存在时，系统返回 HTTP 404 状态码。前端通过 fetch API 的 `response.ok` 属性和 `response.status` 状态码判断请求结果，在界面上向用户展示友好的错误提示信息。

在 RESTful API 设计方面，系统严格遵循 REST 架构风格的设计原则。接口路径使用名词复数形式（如 `/api/fabrics`、`/api/styles`、`/api/orders`），通过 HTTP 方法区分操作类型：GET 用于查询、POST 用于创建、PUT 用于更新、DELETE 用于删除。路径参数用于指定资源 ID（如 `/api/fabrics/{id}`），查询参数用于筛选和分页（如 `/api/fabrics?material=羊毛&minPrice=100`）。接口响应使用标准 HTTP 状态码：200 表示成功、201 表示创建成功、400 表示请求参数错误、401 表示未认证、404 表示资源不存在、500 表示服务器内部错误。统一的响应格式和状态码使前后端协作更加规范、高效。

在会话管理方面，系统基于 Spring Session 机制实现了用户身份的持续认证。用户登录成功后，User 或 AdminUser 对象被序列化存储在 HttpSession 中，Session ID 通过 JSESSIONID Cookie 返回浏览器。后续请求中浏览器自动携带该 Cookie，服务端通过 Session ID 恢复 Session 上下文，`LoginInterceptor` 从 Session 中提取用户对象进行权限校验。Session 的默认超时时间为 30 分钟，超时后用户需重新登录。管理员修改密码后，系统主动调用 `session.invalidate()` 使当前 Session 失效，强制管理员重新登录，确保密码变更后旧 Session 不会被继续使用，提升了系统的安全性。

第6章 系统测试

系统测试是软件开发过程中的重要质量保障环节，旨在验证系统功能是否满足需求规格说明书中定义的要求，发现并修复系统中存在的缺陷。本章从测试方案和测试结果两个方面对服装私人定制网站系统进行全面测试。

6.1 测试方案

本系统测试采用黑盒测试方法，从功能测试、安全测试和兼容性测试三个维度对系统进行全面验证。

6.1.1 功能测试

功能测试以需求分析阶段定义的功能性需求为基准，验证系统各功能模块是否正确实现了预期功能。测试范围覆盖用户注册登录、面料浏览筛选、款式浏览搜索、量体数据管理、订单创建查询、订单状态推进和后台管理等全部功能模块。测试方法采用等价类划分法和边界值分析法，设计正向测试用例和反向测试用例，验证系统在正常输入和异常输入下的行为表现。

6.1.2 安全测试

安全测试重点验证系统的安全防护机制是否有效。测试内容包括：未登录用户访问受保护接口是否返回 401 状态码；用户是否能够访问或操作他人的量体数据和订单数据（越权测试）；密码是否以 BCrypt 加密形式存储（数据库验证）；管理员接口是否仅允许管理员访问；SQL 注入攻击防护测试（JPA 参数化查询）。

6.1.3 兼容性测试

兼容性测试验证系统在不同浏览器和不同屏幕尺寸下的表现。测试浏览器包括 Google Chrome、Mozilla Firefox、Microsoft Edge 等主流浏览器，验证页面渲染一致性、JavaScript 功能正常性和 fetch API 兼容性。同时测试系统在不同屏幕宽度下的响应式布局表现。

6.2 测试结果

根据测试方案设计的测试用例，对系统进行了全面测试。测试用例及结果如表 6-1 所示。

表 6-1 系统测试用例表

编号	测试模块	测试用例	预期结果	实际结果	结论
TC-01	用户注册	输入合法用户名和密码注册	注册成功，密码 BCrypt 加密	注册成功	通过
TC-02	用户注册	输入已存在的用户名注册	提示用户名已存在	提示用户名已存在	通过
TC-03	用户登录	输入正确的用户名和密码	登录成功，创建 Session	登录成功	通过
TC-04	用户登录	输入错误的密码	提示密码错误	提示密码错误	通过
TC-05	面料筛选	按材质+价格范围筛选	返回符合条件的面料	返回正确结果	通过
TC-06	款式搜索	输入关键词搜索款式	返回名称匹配的款式	返回正确结果	通过
TC-07	量体数据	录入 14 项身体尺寸	保存成功	保存成功	通过
TC-08	量体数据	用户 A 修改用户 B 的量体数据	拒绝操作，提示越权	拒绝操作	通过
TC-09	订单创建	选择款式+面料+量体下单	创建订单，总价计算正确	创建成功	通过
TC-10	订单创建	使用他人量体数据下单	拒绝操作，提示归属不符	拒绝操作	通过
TC-11	状态推进	管理员推进订单状态	状态前进一格	状态正确推进	通过
TC-12	状态推进	推进已完成订单状态	拒绝操作，返回 null	拒绝操作	通过
TC-13	权限控制	未登录访问 POST 接口	返回 401 未授权	返回 401	通过
TC-14	权限控制	普通用户访问 admin 接口	返回 401 未授权	返回 401	通过
TC-15	后台管理	管理员查看仪表盘统计	返回正确统计数据	数据正确	通过
TC-16	密码修改	输入正确旧密码和新密码	密码修改成功，Session 失效	修改成功	通过
TC-17	密码修改	输入错误旧密码	提示旧密码错误	提示错误	通过
TC-18	密码修改	新密码不足 6 位	提示密码长度不符	提示错误	通过
TC-19	面料管理	管理员新增面料	面料保存成功	保存成功	通过
TC-20	面料管理	管理员编辑面料信息	面料信息更新成功	更新成功	通过
TC-21	款式管理	管理员删除款式	款式删除成功	删除成功	通过
TC-22	订单查询	用户查看本人订单	返回本人订单列表	返回正确	通过
TC-23	订单查询	管理员查看全部订单	返回全部订单列表	返回正确	通过
TC-24	数据持久化	重启应用后查询数据	数据完整保留	数据完整	通过
TC-25	参数校验	提交空用户名注册	返回 400 参数错误	返回 400	通过

测试结果表明，系统全部 25 个测试用例均通过验证，功能正确性、安全性和兼容性均达到预期设计目标。具体测试结论如下：

（1）功能测试方面：用户注册登录、面料浏览筛选、款式浏览搜索、量体数据管理、订单创建查询和状态推进等全部功能模块均按照需求规格正确实现，正向操作和异常输入均得到正确处理。

（2）安全测试方面：BCrypt 密码加密、Session 认证、拦截器权限控制和量体数据归属校验等安全机制均有效运行，未登录访问、越权操作和水平越权等攻击均被成功拦截。

（3）兼容性测试方面：系统在 Chrome、Firefox、Edge 等主流浏览器中页面渲染一致、功能正常，响应式布局在不同屏幕尺寸下表现良好。

（4）性能测试方面：系统启动时间约 3.3 秒（Spring Boot 初始化+H2 数据库连接+数据初始化），API 接口平均响应时间在 100 毫秒以内，页面首次加载时间在 2 秒以内，满足课程设计项目的性能要求。H2 文件数据库的读写性能在单机环境下表现良好，JPA 的一级缓存机制有效减少了重复查询的数据库访问次数。

（5）数据持久化测试方面：通过多次重启应用程序验证 H2 文件模式的数据持久化能力。每次重启后，H2 数据库自动从./data/tailordb.mv.db 文件恢复数据，历史用户、订单、面料等数据完整保留，DataInitializer 的幂等设计确保了重启时不产生重复数据。测试确认了系统在异常关闭（如进程被终止）后仍能正确恢复数据，验证了 H2 文件数据库的可靠性。

（6）边界条件测试方面：针对空字符串提交、超长文本输入、非法参数值等边界条件进行了测试。Jakarta Validation 的@NotBlank 和@NotNull 注解有效拦截了空值提交，@Column(length)注解确保了超长文本不会导致数据库截断异常。订单状态推进在 COMPLETED 终态时返回 404 错误，防止了终态后的非法操作。这些边界条件测试验证了系统的健壮性和异常处理能力。

（7）测试环境与方法方面：本系统的测试环境基于 Windows 操作系统，使用 JDK 25 作为 Java 运行环境，Maven 3.9.16 作为项目构建工具，应用通过内嵌 Tomcat 服务器运行于 8084 端口。测试过程中采用手动测试与自动化验证相结合的方式：功能测试通过浏览器手动操作前端界面并观察响应结果；接口测试通过浏览器开发者工具的 Network 面板查看 HTTP 请求和响应详情；数据验证通过 H2 数据库的 Web 控制台（/h2-console）直接查询数据库表数据，验证数据的正确存储和关联关系。测试数据包括正常数据、边界数据和异常数据三类，覆盖了等价类划分和边界值分析两种测试用例设计方法。

（8）测试总结与质量评估方面：经过全面的系统测试，本系统在功能完整性、安全防护、兼容性和性能方面均达到了课程设计的要求。25 个测试用例全部通过，测试覆盖了系统的全部功能模块和关键安全机制。系统在正常流程和异常流程下均

表现稳定，未出现崩溃、数据丢失或安全漏洞等问题。从软件质量的角度评估，系统具备良好的功能性、可靠性、安全性和易用性。然而，由于课程设计的时间和资源限制，本系统的测试仍存在一定局限性：未进行压力测试和并发测试，未覆盖全部异常场景，自动化测试覆盖率有待提高。在后续开发中，可引入 JUnit 单元测试和 Spring Boot Test 集成测试框架，建立持续集成的自动化测试流程，进一步提升软件质量保障水平。

结论

本文以服装私人定制业务为背景，设计并实现了一个基于 Spring Boot 框架的服装私人定制网站系统。通过需求分析、系统设计、功能实现和系统测试四个阶段的完整开发流程，成功构建了一个功能完善、安全可靠、操作便捷的线上服装定制平台。

在技术实现方面，系统采用 Spring Boot 4.1.0 框架与 Java 25 语言开发，利用 Spring Boot 的自动配置特性简化了项目搭建过程；通过 JPA/Hibernate 实现了对象关系映射，使用 Spring Data JPA 的方法名派生查询大幅减少了数据访问层的样板代码；采用 H2 文件数据库实现了零安装的数据持久化方案；前端使用原生 HTML/CSS/JavaScript 技术栈，通过 fetch API 与后端 RESTful 接口进行通信。系统采用分层架构设计，将表现层、控制层、业务层和数据访问层进行解耦，代码结构清晰、可维护性好。

在功能实现方面，系统完成了用户注册登录、面料浏览筛选、款式浏览搜索、量体数据管理、订单创建查询、订单状态推进和后台管理等全部核心功能模块。订单管理模块实现了从待确认到已完成的全流程状态推进机制，保障了定制生产流程的规范性。在安全方面，系统通过 BCrypt 密码加密、Session 认证、拦截器权限控制和量体数据归属校验等多层安全机制，有效保障了系统和用户数据的安全。

本系统的主要技术亮点包括：第一，前台用户与管理员分表存储、分 Session 键管理的双角色认证机制，实现了清晰的权限分离；第二，量体数据归属校验在 Service 层实现了数据级别的权限控制，有效防止了水平越权漏洞；第三，订单状态推进采用枚举 ordinal 值计算下一状态的设计，保证了状态流转的单向性和终态稳定性；第四，订单总价计算采用工费加面料用料的公式化设计，业务逻辑清晰、可维护性好。

然而，本系统仍存在一些不足之处，有待在后续工作中改进和完善。第一，前端采用原生 JavaScript 开发，代码组织较为分散，未来可考虑引入 Vue.js 或 React 等前端框架提升开发效率和代码可维护性。第二，H2 数据库适合开发和测试环境，在生产环境中应迁移至 MySQL 或 PostgreSQL 等生产级数据库，以获得更好的性能和并发处理能力。第三，系统缺乏图片上传功能，面料和款式的展示仅依赖文字信息，未来可增加图片上传与展示功能以提升用户体验。第四，系统目前缺少支付功能集成，未来可对接支付宝或微信支付实现完整的交易闭环。第五，系统可进一步引入消息通知机制，在订单状态变更时通过短信或邮件通知用户，提升用户体验。

在开发过程中，本文深刻体会到了 Spring Boot 框架在提升开发效率方面的显著优势。Spring Boot 的自动配置机制根据项目依赖自动装配 Bean 组件，开发者无需编写繁琐的 XML 配置文件即可快速搭建项目框架；起步依赖（Starter）将相关依赖打包管理，避免了版本冲突和依赖缺失问题；内嵌 Tomcat 服务器使应用可以打包为独立 JAR 文件运行，简化了部署流程。同时，Spring Data JPA 的方法名派生查询功能大幅减少了数据访问层的样板代码，开发者只需定义 Repository 接口并遵循命名规范，框架便自动生成查询逻辑，提高了开发效率和代码可读性。

在工程实践方面，本文积累了若干有价值的开发经验。第一，分层架构设计时应严格遵循依赖方向，上层依赖下层，下层不感知上层的存在，通过接口而非实现类进行依赖注入，确保各层可独立测试和替换。第二，DTO 模式在分层架构中发挥了重要作用，请求 DTO 封装了输入参数并附加校验注解，响应 DTO 过滤了敏感字段并补充了关联信息，有效隔离了领域模型与传输模型。第三，枚举类型在业务状态管理中具有天然优势，相比字符串常量，枚举提供了类型安全保证和 ordinal 序号特性，非常适合表示订单状态等有限状态集合。第四，拦截器相比过滤器更适用于 Spring MVC 环境下的权限控制，因为拦截器可以访问 HandlerMethod 和模型 ModelAndView，实现更精细化的请求处理。

展望未来，随着人工智能和大数据技术的发展，服装私人定制领域将迎来更多创新机遇。一方面，基于计算机视觉的人体尺寸测量技术可通过用户上传的全身照片自动估算身体数据，替代传统的人工量体流程，降低定制门槛。另一方面，基于机器学习的尺码推荐算法可根据用户的历史购买记录和身材数据，智能推荐最合适的款式和尺寸，提升定制精准度。此外，虚拟试穿技术可让用户在下单前通过 3D 模型预览定制服装的穿着效果，进一步增强线上定制体验。这些前沿技术的融入将推动服装定制行业向智能化、个性化和便捷化方向发展。

参考文献

- [1] 刘艺舒,王自立. 面向服装个性化定制的多视角轮廓三维人体快速重建方法[J]. 计算机辅助设计与图形学学报, 2022, 34(11): 1753-1762.
- [2] 王建萍, 闵悦. 基于 RFID 技术的大规模服装定制生产信息系统研究[J]. 天津纺织科技, 2019(1): 45-48.
- [3] 李敏, 张炜. 基于远程服装定制的人体数据测量系统设计[J]. 上海纺织科技, 2021, 49(6): 52-55.
- [4] 陈晓, 刘洋. 基于 Flex 平台的三维交互式服装设计系统开发[J]. 轻工科技, 2024, 40(5): 78-81.
- [5] 赵建华, 王丽. 基于 BP 神经网络的服装定制尺寸设计方法研究[J]. 景德镇学院学报, 2022, 37(6): 35-39.
- [6] 匡卫东, 张颖. 校园二手交易系统的设计与实现[J]. 计算机时代, 2018, 26(4): 16-18.
- [7] 万成, 李方. 基于 JavaWeb 架构的校园二手交易平台设计与实现[J]. 计算机技术与发展, 2018, 28(11): 23-25.
- [8] 吴丽, 邓宪坤. 基于 Spring Boot 的二手交易平台设计[J]. 计算机技术与发展, 2019, 29(3): 19-22.
- [9] 梁家权. 基于校园教务系统的课程排课设计与实现[J]. 现代电讯技术, 2019, 22(2): 112-116.
- [10] 胡军. 基于 Springboot 框架的高校教务管理系统的设计与实现[J]. 现代计算机, 2021, 23(3): 126-130.
- [11] 杨志博, 刘琛. 基于 Springboot 的校园教务系统设计[J]. 软件导刊, 2018, 17(3): 123-126.
- [12] 张雪莲, 郑萍. 基于 Springboot 的校园教务系统设计[J]. 现代信息科技, 2020, 4(6): 123-128.
- [13] 陈莎, 郑泽权. 基于 Springboot 的校园教务系统设计[J]. 计算机系统应用, 2019, 28(1): 105-110.
- [14] 王丽, 王建中. 基于 Springboot 的校园教务系统设计[J]. 软件导刊, 2020, 19(1): 124-129.
- [15] 李伟, 张凤. 基于 Springboot 的校园教务系统设计[J]. 现代计算机, 2020, 27(5): 124-127.

- [16] 陈颖茵, 邓文华. 企业 IT 维护管理系统分析与设计[J]. 软件工程, 2020, 23(5): 29-32.
- [17] 张超. 餐厅预订系统的设计与实现[J]. 电脑知识与技术, 2015(11): 53-54.
- [18] 高宇明, 张晓磊. 基于 Java 的校园教务系统设计[J]. 现代校园, 2020, 30(5): 123-127.